

A sound pure type system with ranks as sorts and Type-in-Type

Pavel Sokolov

October 9, 2026

Abstract

We present a dependently typed pure type system which contains a circular axiom $s : s$ for a certain sort s and is nonetheless strongly normalizing. Girard’s paradox is avoided because our sorts are not closed under a Π -former: function types are placed in sorts according to their *rank*. We prove strong normalization of a system and provide barebones set-theoretical model for it. A normalization proof is surprisingly simple (modulo general metatheoretical properties of pure type systems) and has been fully formalized in vanilla Rocq (formerly Coq) without any additional axioms.

1 Introduction

Pure type systems (PTSs) are a family of minimalistic typed λ -calculi including grammar formers for variables $x \in \mathbf{Var}$, abstraction $\lambda(x : T).t$, application $t t'$, types of dependent functions $\Pi(x : T).U$ (where U might depend on x) and *sorts* $s \in \mathcal{S}$ which act like “types of types”. In contrast with simpler type systems, there is no syntactical distinction between terms and types — they are defined by a single grammar; an expression can act both as a term and as a type depending on a side where it stands next to the colon in the typing judgement $\Gamma \vdash _ : _$. A single PTS is determined by two relations:

- *axiom* relation $\mathbf{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ determines “to which sort(s) a given sort belongs as an element”; instead of $(s, s') \in \mathbf{Ax}$ it is customary to write simply $s : s'$.
- *rule* relation $\mathbf{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ places types of functions $\Pi(x : T).U$ into sorts accounting for the sorts where T and U are situated.

Systems from Barendregt’s lambda cube fit naturally into the framework: for example, simply typed lambda calculus (STLC) can be viewed as a pure type system with a single sort $*$, no axioms and a single rule $(*, *, *)$. In general, however, a pure type system is dependently typed since Π allows the codomain

$$\begin{array}{c}
\text{Ax} \\
\frac{}{\Gamma \vdash s : s'} (s, s') \in \text{Ax}
\end{array}
\quad
\begin{array}{c}
\text{Rule} \\
\frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash U : s_U}{\Gamma \vdash \Pi(x : T).U : s} (s_T, s_U, s) \in \text{Rule}
\end{array}$$

$$\begin{array}{c}
\text{Var} \\
\frac{}{\Gamma \vdash x : T} x : T \in \Gamma
\end{array}
\quad
\begin{array}{c}
\text{Abs} \\
\frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash u : U}{\Gamma \vdash \lambda(x : T).u : \Pi(x : T).U}
\end{array}$$

$$\begin{array}{c}
\text{App} \\
\frac{\Gamma \vdash u : \Pi(x : T).U \quad \Gamma \vdash t : T}{\Gamma \vdash ut : U[t/x]}
\end{array}
\quad
\begin{array}{c}
\text{Conv} \\
\frac{\Gamma \vdash t : T \quad \Gamma \vdash U : s \quad T =_{\beta} U}{\Gamma \vdash t : U}
\end{array}$$

Figure 1: Typing rules of pure type systems

type to depend on the value of an argument — Coquand’s Calculus of Constructions (CoC) sits comfortably at the top of lambda cube and is modeled as a pure type system with two sorts $\{*, \square\}$, a single axiom $* : \square$ and four rules corresponding to simple functions $(*, *, *)$, polymorphic functions $(\square, *, *)$, type operators $(\square, \square, \square)$ and dependent types $(*, \square, \square)$.

As is known, both STLC and CoC are strongly normalizing and sound with respect to their set-theoretical semantics. A classical result shows that it is not true for arbitrary PTS.

Theorem (Girard’72). *The following PTS called System U is not normalizing:*

$$\begin{aligned}
\mathcal{S} &= \{*, \square, \triangle\}, & \text{Ax} &= \{(*, \square), (\square, \triangle)\}, \\
\text{Rule} &= \{(*, *, *), (\square, *, *), (\triangle, *, *), (\square, \square, \square), (\triangle, \square, \square)\}
\end{aligned}$$

*Futhermore, the offending term has type $\Pi(P : *).P$.*

This result is dubbed “Girard’s paradox” since it renders System U inconsistent with respect to its intended logical semantics: inhabitedness of $\Pi(P : *).P$ would mean that every proposition is provable. Thus any system where System U embeds is inconsistent, too; in particular, an original version of Martin-Löf’s Intuitionistic Type Theory embeds the following PTS known as “Type-in-Type” which is inconsistent due to Girard’s paradox:

$$\mathcal{S} = \{*\}, \quad \text{Ax} = \{* : *\}, \quad \text{Rule} = \{(*, *, *)\}.$$

For a pure type system with usual set-theoretical semantics, it is natural to expect that a sort of *small types* $*$ is closed under Π , thus we cannot forbid the rule $(*, *, *)$; then, a circular axiom $* : *$ has to go. Long story short, it motivates an introduction of *universe hierarchy* $\mathcal{S} = \{\mathcal{U}_k\}_{k \in \mathbb{N}}$ with axioms $\mathcal{U}_k : \mathcal{U}_{k+1}$ and rules $(\mathcal{U}_k, \mathcal{U}_l, \mathcal{U}_{k \vee l})$ for each k, l ; when interpreted in sets, $\llbracket \mathcal{U}_k \rrbracket$ is taken to be the k -th inaccessible cardinal.

Observing this, we took interest in the following two questions:

- If we were to drop the rule $(*, *, *)$ and keep the axiom $* : *$ instead, can we find a nontrivial consistent pure type system?
- Is there a consistent pure type system with total relations **Ax** and **Rule** whose natural set-theoretical semantics do not involve assumption of inaccessible cardinals?

Turns out that, for both questions, answer is “yes”. Furthermore, both yes-s are witnessed by a single PTS, which we dub **PTS-*rk***:

$$\mathcal{S} = \{\mathbf{rk}_r\}_{r \in \mathbb{N}} \quad \mathbf{Ax} = \{(\mathbf{rk}_r, \mathbf{rk}_0)\}_{r \in \mathbb{N}} \quad \mathbf{Rule} = \{(\mathbf{rk}_r, \mathbf{rk}_s, \mathbf{rk}_{(r+1) \vee s})\}_{r, s \in \mathbb{N}}$$

(Naming “**rk**” comes from *rank*, since rules compute precisely the rank of functions, which is the maximal nesting of Π in the domain type.)

Indeed, there is a circular axiom $\mathbf{rk}_0 : \mathbf{rk}_0$, and System U is not embeddable since $\mathbf{rk}_r$ is not closed under Π for any r . Also, as we will see in Section 3, our **PTS-*rk*** is strongly normalizing and has a set-theoretical model in ZFC despite an intuition that $\mathbf{rk}_r : \mathbf{rk}_0$ might “feel wrong”. Modulo general metatheory of pure type systems laid out in Section 2, a normalization proof turned out to be suspiciously simple, even more so in comparison with normalization proofs of other dependently typed systems. To dispel our own doubts, we formalized a proof of normalization in Rocq (formerly Coq) without using any additional axioms, yielding a fully constructive proof; our journey in formalization is summarised in Section 4. We provide a closing summary and some pointers on possible extensions of the system in Section 5.

2 Metatheory of pure type systems

For ease of presentation of our results, in this and the following section we take a Barendregt convention which assumes that all bound variables are distinct from free variables and from each other; furthermore, α -equivalence (renaming of bound variables) is assumed as a trivial equality on terms. Nevertheless, our formalization takes variable treatment seriously; see Section 4 for more details.

In Fig. 1, a rule of *conversion* (CONV) mentions *definitional equivalence* of types $T =_\beta U$. It is defined as a reflexive, symmetric, transitive closure of a one-step reduction.

Definition 1 (One-step reduction). Relation of a one-step reduction $t \rightarrow_\beta t'$ is the smallest relation satisfying the following rules:

$$\begin{array}{c} \frac{T \rightarrow_\beta T'}{\Pi(x : T).U \rightarrow_\beta \Pi(x : T').U} \qquad \frac{U \rightarrow_\beta U'}{\Pi(x : T).U \rightarrow_\beta \Pi(x : T).U'} \\[10pt] \frac{T \rightarrow_\beta T'}{\lambda(x : T).t \rightarrow_\beta \lambda(x : T').t} \qquad \frac{t \rightarrow_\beta t'}{\lambda(x : T).t \rightarrow_\beta \lambda(x : T).t'} \qquad \frac{t \rightarrow_\beta t'}{tu \rightarrow_\beta t'u} \\[10pt] \frac{u \rightarrow_\beta u'}{tu \rightarrow_\beta tu'} \qquad \text{BETA} \qquad (\lambda(x : T).t)u \rightarrow_\beta t[u/x] \end{array}$$

In essence, actual reduction happens only in rule BETA; other rules simply state that we can freely reduce any subterm.

The following result will be greatly useful in our development. It was proven many times over for different similar systems; we briefly revisit the proof since previous presentations usually did not allow reductions in the type of argument T in $\lambda(x : T).t$.

Theorem 1 (Characterization of $(=_{\beta})$). *In arbitrary PTS,*

$$t =_{\beta} u \quad \Leftrightarrow \quad \exists v. t \rightarrow_{\beta}^* v \wedge u \rightarrow_{\beta}^* v,$$

where $(\rightarrow_{\beta}^*)$ is reflexive transitive closure of $(\rightarrow_{\beta})$.

Proof. Note that it is enough to prove that $(\rightarrow_{\beta}^*)$ has a diamond property — for each t_0, t_1, t_2 such that $t_0 \rightarrow_{\beta}^* t_1$ and $t_0 \rightarrow_{\beta}^* t_2$ there exists a t' such that $t_1 \rightarrow_{\beta}^* t'$ and $t_2 \rightarrow_{\beta}^* t'$:

$$\begin{array}{ccc} t_0 & \longrightarrow & t_1 \\ \downarrow & & \downarrow \\ t_2 & \dashrightarrow & t' \end{array}$$

Then, any “mountains” in the path $t =_{\beta} u$ can be switched into “valleys”.

Now, diamond property is proven by standard technique of parallel reduction defined as the smallest relation satisfying:

$$\begin{array}{c} t \Rightarrow t \\ \frac{T \Rightarrow T' \quad U \Rightarrow U'}{\Pi(x : T).U \Rightarrow \Pi(x : T').U'} \quad \frac{T \Rightarrow T' \quad t \Rightarrow t'}{\lambda(x : T).t \Rightarrow \lambda(x : T').t'} \\[10pt] \frac{t \Rightarrow t' \quad u \Rightarrow u'}{tu \Rightarrow t'u'} \quad \frac{t \Rightarrow t' \quad u \Rightarrow u'}{(\lambda(x : T).t)u \Rightarrow t'[u'/x]} \end{array}$$

Intuitively, while $(\rightarrow_{\beta})$ is a single β -reduction somewhere in the term and $(\rightarrow_{\beta})^*$ is a sequence of reductions, $(\Rightarrow)$ is several simultaneous reductions immediately available in the term. Notice, then, that $(\rightarrow_{\beta}) \subseteq (\Rightarrow) \subseteq (\rightarrow_{\beta}^*)$; by monotonicity of reflexive transitive closure, $(\rightarrow_{\beta}^*) \subseteq (\Rightarrow^*) \subseteq (\rightarrow_{\beta}^*)^* = (\rightarrow_{\beta}^*)$, so $(\Rightarrow^*) = (\rightarrow_{\beta}^*)$. Thus it is enough to prove diamond property of $(\Rightarrow^*)$. Diamond property is preserved under reflexive transitive closure, so it is enough to prove diamond property of $(\Rightarrow)$, which holds by tedious case analysis. $\square$

Setting aside practical applications of this theorem (to algorithmically check equivalence of terms, we can now search for common redex), it will prove useful already in our proof development. For example, the following “equality inversion” lemmas are direct corollaries:

Corollary 1 (Equality inversion). *If $s =_{\beta} s'$, then $s = s' \in \mathcal{S}$; if $\Pi(x : T).U =_{\beta} \Pi(x : T').U'$, then $T =_{\beta} T'$ and $U =_{\beta} U'$.*

They will become necessary in proving subject reduction, or type preservation. But first, we will show some structural properties of a typing judgement given in Fig. 1. Again, they surely have been proven before for PTS, but we slightly differ in the formulation of typing rules in important places so it's better to recheck that these hold.

Lemma 1 (Weakening). *The following rule is admissible:*

$$\frac{\Gamma, \Delta \vdash t : T}{\Gamma, x : U, \Delta \vdash t : T}.$$

Proof. By structural induction on the typing derivation. $\square$

Lemma 2 (Substitution). *The following rule is admissible:*

$$\frac{\Gamma, x : U, \Delta \vdash t : T \quad \Gamma \vdash u : U}{\Gamma, \Delta[u/x] \vdash t[u/x] : T[u/x]}.$$

Proof. By structural induction on the typing derivation for t . For variables in Δ , it is necessary that both Δ and T get substitutions of u for x so VAR is still applicable; for places in t where x used to be we need to apply weakening lemma to the typing derivation for u exactly $|\Delta|$ times. In the case of CONV it is important that definitional equality of types is preserved by substitution (thanks to characterization of $(=_{\beta})$). $\square$

In addition, a family of so-called *inversion lemmas* stating that type of well-typed term is largely determined by shape of a term are also proven by straightforward induction on typing derivation (each one is proven separately, no mutual induction involved):

Lemma 3 (Inversion lemmas).

$$\Gamma \vdash s : T \Rightarrow \exists s'. (s, s') \in \mathbf{Ax} \wedge T =_{\beta} s' \quad (1)$$

$$\Gamma \vdash \Pi(x : T).U : V \Rightarrow \exists s_T, s_U, s. \begin{cases} (s_T, s_U, s) \in \mathbf{Rule} \\ \Gamma \vdash T : s_T \\ \Gamma, x : T \vdash U : s_U \\ V =_{\beta} s \end{cases} \quad (2)$$

$$\Gamma, x : T, \Delta \vdash x : U \Rightarrow T =_{\beta} U \quad (3)$$

$$\Gamma \vdash \lambda(x : T).t : V \Rightarrow \exists s, U. \begin{cases} \Gamma \vdash T : s \\ \Gamma, x : T \vdash t : U \\ V =_{\beta} \Pi(x : T).U \end{cases} \quad (4)$$

$$\Gamma \vdash tu : V \Rightarrow \exists U, T. \begin{cases} \Gamma \vdash t : \Pi(x : U).T \\ \Gamma \vdash u : U \\ V =_{\beta} T[u/x] \end{cases} \quad (5)$$

To progress further, we need to make certain assumptions on the PTS we have in question.

Definition 2 (Functional PTS). A pure type system is called *functional* if $\text{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\text{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are functional relations, that is,

$$(s : s') \wedge (s : s'') \Rightarrow s' = s'' \quad \text{and} \quad (s_1, s_2, s) \in \text{Rule} \wedge (s_1, s_2, s') \in \text{Rule} \Rightarrow s = s'$$

Definition 3 (Total PTS). A pure type system is called *total* if $\text{Ax} \subseteq \mathcal{S} \times \mathcal{S}$ and $\text{Rule} \subseteq \mathcal{S}^2 \times \mathcal{S}$ are total relations, that is,

$$\forall s. \exists s'. (s, s') \in \text{Ax} \quad \text{and} \quad \forall s_1, s_2. \exists s. (s_1, s_2, s) \in \text{Rule}$$

Theorem 2 (Uniqueness of typing). *In a functional PTS, the following rule is admissible:*

$$\frac{\Gamma \vdash t : T \quad \Gamma \vdash t : T'}{T =_{\beta} T'}.$$

Proof. By structural induction on t and inversion of given typing derivations.

- $t = s \in \mathcal{S}$: by functionality of Ax .
- $t = \Pi(x : T).U$: inversion of typing gives two (possibly different) sorts for T and for U . By induction we get that they're pairwise definitionally equivalent, which by equality inversion gives strictly equal sorts. Then what is needed holds by functionality of Rule .
- Case for $t = x \in \text{Var}$ holds trivially.
- $t = (\lambda(x : T).u)$: by induction hypotheses and by congruence of $(=_{\beta})$.
- $t = uv$: by typing inversion and by induction hypothesis for u we get $u : \Pi(x : T_1).U_1$ and $\Pi(x : T_1).U_1 =_{\beta} \Pi(x : T_2).U_2$. We need $U_1[v/x] =_{\beta} U_2[v/x]$ which we get from equality inversion and preservation of def.eq. under substitution.

□

In our proof of normalization, we're going to rely on yet another property: if there is a derivation for $t : T$, then T is itself typeable with some sort. However, it doesn't hold in our current formulation of typing rules: in $\Gamma \vdash x : T$, nothing prevents a context Γ to contain an entry $(x : T')$ where T' is not typeable. Thus we need an additional assumption.

Definition 4 (Context well-formedness). Context Γ is *well-formed* (written $\vdash \Gamma$) if every type in it is typed by some sort. Formally, set of well-formed contexts is the least set closed by the following rules:

$$\frac{}{\vdash \cdot} \quad \text{and} \quad \frac{\vdash \Gamma \quad \Gamma \vdash T : s}{\vdash \Gamma, x : T}$$

(where $\cdot$ denotes an empty context.)

Usually, when presenting a dependently typed system, context well-formedness is added as a premise to typing rules which would otherwise be premise-free (such as AX and VAR in a PTS). Here we instead provide it as an additional assumption only when necessary; in addition to leading to simpler metatheory (we avoid an otherwise mutually inductive definition of $(\vdash _)$ and $(_ \vdash _ : _)$), this is closer to how decision procedures for typing judgements are implemented.

Theorem 3 (Typing yields correct type). *In a total PTS, if $\Gamma \vdash t : T$ and $\vdash \Gamma$, then there exists $s \in \mathcal{S}$ such that $\Gamma \vdash T : s$.*

Proof. By structural induction on typing derivation.

- AX, RULE: by totality of Ax.
- VAR: follows from $\vdash \Gamma$.
- ABS: by totality of Rule.
- APP: $\Gamma \vdash tu : T[u/x]$. From induction hypothesis for t we know that $\Gamma \vdash t : \Pi(x : U).T : s$ for some s , then by typing inversion for $\Pi(x : U).T$ we get that T is typable with some sort. Then what is needed follows from substitution lemma.
- CONV: what is needed is present in the premise.

□

Totality is not a particularly strict requirement — similar true propositions can be stated for non-total PTS, for example in CoC typing yields type which is either itself typable or equal to $\square$. But PTS-*rk* is indeed total so the theorem above is enough for our purposes.

3 Soundness of PTS-rk

4 Formalizing normalization of PTS-rk in Rocq

Inductive characterization of the set of strongly normalizing terms follows [1].

5 Conclusion

References

- [1] Felix Joachimski and Ralph Matthes. “Short proofs of normalization for the simply-typed λ -calculus, permutative conversions and Gödel’s T”. In: *Archive for Mathematical Logic* 42.1 (2003), pp. 59–87. DOI: 10.1007/s00153-002-0156-9. URL: <https://doi.org/10.1007/s00153-002-0156-9>.